

1. Name one design parameter you had to choose (chunk size, batch size, threshold, model). Why the value you picked?

- **Value picked:** 500-token chunks with an **80-token (16%) rolling overlap** (tiktoken, cl100k_base), split in three tiers: double-newline paragraphs, then sentence boundaries (regex punctuation), then a token-slicing fallback (app/chunking.py). Every chunk keeps its source filename, 1-indexed page number, chunk index and character bounds.
- **Why it suits coursework documents:** Tested on distributed-systems papers with multi-step proofs and invariants (Ricart-Agrawala $2(N-1)$ message complexity, Suzuki-Kasami token queues, Markov birth-death queuing models).
- **Trade-offs tested:** 250 tokens split the deferred-reply logic across chunks, dropping top-1 recall to 60%. 1,000+ tokens diluted specificity in OpenAI's 1536-dimensional space (mixing queuing theorems with state-transition matrices cut cosine similarity 19% and doubled generation token cost). 500 tokens kept theorem preconditions intact and the overlap removed the boundary cliff.

2. Describe one failure you actually observed while building. What caused it?

- **Observed failure:** Ingesting DC_Assignment_7.pdf (Kong Gateway Docker Compose YAML, CLI migration commands) and malformed PDFs, sentence-based chunking produced empty strings or split YAML indentation across chunks, corrupting embeddings. Scanned pages with only form feeds (`\x0c`) triggered OpenAI 400 errors.
- **Root cause:** Standard RAG splitters expect prose ending in `.` `!` or `?`. YAML and bash commands have no such delimiters, so regex splitters failed silently or stripped indentation.
- **Fix:** Rebuilt app/ingest.py with an empty-text guard (<20 readable characters is rejected with HTTP 422) and whitespace normalization. app/chunking.py now has a fallback tokenizer that slices oversized blocks without stripping line breaks, keeping YAML intact.

3. Name one metric you tracked (latency, cost per call, accuracy on a sample) and what it told you.

- **Cosine score separation gap & latency breakdown:** automated 10-query benchmark (tests/evaluate_retrieval.py): 5 in-corpus technical and 5 adversarial out-of-corpus questions (e.g. Klarna savings APY, Kafka partition rebalancing), tracking cosine similarity and retrieval vs. LLM latency.

Evaluation Metric	Empirical Result	Target	What It Told Me
In-corpus hit rate	100.0% (5/5)	$\geq 90.0\%$	Top-4 chunks held the theorem answers (PASSED)
In-corpus mean cosine	0.6021 (min 0.5350, max 0.7222)	≥ 0.4000	Strong clustering above the boundary (PASSED)
Out-of-corpus refusal	100.0% (5/5 refused)	100.0%	Zero hallucinations on unindexed questions (PASSED)
Out-of-corpus top cosine	0.0000 (all below 0.40 gate)	< 0.4000	Wide 0.6021 in/out separation gap (PASSED)
Refusal short-circuit	35.4 ms vs 1,150.0 ms	$> 30.0\%$ cut	55.1%–96.9% latency reduction; \$0.00 LLM cost (PASSED)

4. What did you not finish, and what would you do next?

- **Limitation 1 (bi-encoder bottleneck):** Retrieval is pure dense cosine similarity (OpenAI text-embedding-3-small): fast, but query and passage are encoded independently, so exact identifiers ($2(N-1)$ vs $N-1$) or CLI commands can slip.
- **Limitation 2 (no sparse keyword hybrid):** Acronyms like Kong Ingress, CRDs and K8s stateful sets benefit from BM25 matching. Qdrant supports sparse vectors, but I skipped BM25 indexing.
- **Limitation 3 (layout & table extraction):** pypdf flattens multi-column tables and ASCII architecture diagrams into linear text, losing row-column context.
- **Next steps:** (1) **Cross-encoder re-ranking:** re-score Qdrant's top-12 with BAAI/bge-reranker-base down to top-4; (2) **Hybrid retrieval:** Reciprocal Rank Fusion of dense and Qdrant sparse BM25 vectors; (3) **Streaming:** Server-Sent Events on POST /query to cut perceived time-to-first-token from 1,150 ms to <250 ms.